

AutoAIM Studio

User Manual

Version 2.0.0 | October 2026

Contact: veradelagarza@quimica.unam.mx

Last Updated: October 9, 2026

Tested With: 1024+ automated tests (0 failures, 75 skipped)

Table of Contents

1. Introduction
2. What's New in v2.0.0
3. Getting Started
4. Data Tab
5. Training Tab
6. Gaussian Process (v2.0 → v2.1)

- v2.1: Kernel Family Tuning
- Model Provenance

7. Neural Network Tab
8. Optimization Tab
9. Results Tab
10. Ensemble Tab
11. Explainability Tab
12. Predict Tab
13. Zero Data Leakage Pipeline (v2.0)
14. Structure Linking
15. Troubleshooting

Introduction

AutoAIM Studio is a comprehensive machine learning platform designed for materials science applications. It provides an intuitive graphical interface for data preprocessing, model training, hyperparameter optimization, neural network construction, ensemble modeling, and model interpretation.

Key Features

- Link Crystal Structures (v2.0): Connect CIF structure files to tabular datasets via an ID column
- Data Health Check (v2.0): Automated quality analysis before training
- Session Exporter (v2.0): Export complete results in .txt/.md/.csv/.json
- Confidence Intervals (v2.0): Prediction intervals for regression models
- Model Cards (v2.0): Auto-generated documentation for each saved model
- Automated Data Preprocessing: Handle missing values, categorical encoding, and feature scaling
- Custom Composition Feature Engineering: Generate 106 composition-based descriptors from chemical formulas

- Crystal Structure Loading: Import CIF, POSCAR, CONTCAR, VASP, XYZ files (v1.2+)
- Multiple ML Algorithms: Random Forest, Gradient Boosting, Histogram Gradient Boosting, XGBoost, LightGBM, CatBoost, SVR, Ridge, k-Nearest Neighbors, Gaussian Process (v2.1 — full kernel family tuning + native uncertainty quantification)
- Neural Network Builder: Visual architecture design with customizable layers
- Hyperparameter Optimization: Bayesian optimization with Optuna, with parameter selection
- Ensemble Modeling: Combine multiple models with weight optimization
- Model Interpretability: SHAP values, permutation importance, partial dependence plots
- Standalone Prediction Mode: Deploy trained models for inference on new data

System Requirements

- Windows 10/11 (64-bit)
- 8GB RAM minimum (16GB recommended)
- CUDA-compatible GPU (optional, for neural network acceleration)

Tested Configurations

AutoAIM Studio v2.0.0 has been validated with:

- 1024+ automated tests — 0 failures, 75 skipped (optional dependencies)
- 44 test modules covering core functionality, integration workflows, and utilities
- Python 3.10–3.12 on Windows 10/11 and Linux (Ubuntu 22.04)
- PyInstaller 6.2.0 one-directory builds

What's New in v2.0.0

Link Crystal Structures

The most significant addition in v2.0.0 is the ability to link crystal structure files (CIF) to tabular datasets via an ID column. This bridges the gap between tabular machine learning and structural information.

How to use:

16. Prepare your CSV with an ID column that matches your CIF filenames (e.g., material_id)
17. Name CIF files as {ID_value}.cif (e.g., mp-1234.cif)
18. Click "Link Crystal Structures" in the Data tab
19. Select your ID column and CIF folder
20. AutoAIM extracts 149 crystal descriptors and merges them with your dataset

Crystal features extracted (149 total):

- Lattice (10): cell parameters, volume, symmetry flags (cubic/hexagonal)

- Composition-weighted (13): averaged periodic properties (electronegativity, ionization energy, atomic radius, etc.)
- Structural (3): volume per atom, packing fraction, space group number
- Derived (2): density, total electrons
- Magpie composition (106): full statistical descriptors from the CIF formula
- Advanced structural (15): bond lengths, coordination numbers, RDF peaks, structure complexity index, lattice anisotropy

All features use the same computation core as direct CIF loading — only the column prefix differs (crystal_ for linked features).

Polymorph detection: If multiple rows share the same ID, AutoAIM warns you about potential polymorphs before proceeding.

Important: When crystal structures are linked, the Magpie composition featurizer is automatically disabled to avoid duplicate columns. The CIF-derived features already include composition descriptors.

Data Health Check

Before training, run the Data Health Check to analyze your dataset quality:

- Detects missing values, outliers, duplicate rows, constant columns
- Checks scaling mismatches, high correlations, class imbalance
- Provides an overall score (0-100) and actionable recommendations
- Access via the "Data Health Check" button in the Data tab

Session Exporter

Export your complete session with all results:

- 4 formats: .txt (readable), .md (Markdown), .csv (spreadsheet), .json (machine)
- Includes: model parameters, per-fold CV scores, optimization trials, NN history
- Includes learning curves data, confidence intervals, and optimization configuration
- Access via the "Export Session" button in the Results tab

Confidence Intervals

For regression predictions, AutoAIM can compute confidence intervals:

- 68% and 95% intervals based on cross-validation residuals
- Available in the Predict tab via the "Include Confidence Intervals" checkbox
- Works for all model types (sklearn, neural networks, ensembles)

Model Cards

Each saved model now includes an auto-generated MODEL_CARD.md with:

- Model details, intended use, dataset characteristics

- Performance metrics, confidence intervals, feature names
- Limitations and reproducibility instructions

Zero Data Leakage Pipeline (v2.0 CRITICAL FIX)

AutoAIM Studio now uses sklearn Pipeline internally to completely eliminate data leakage during cross-validation:

The problem (now fixed): Previously, StandardScaler was fitted on the entire training set before cross-validation. This meant every validation fold's statistics leaked into the scaler, producing slightly over-optimistic CV scores.

The fix:

- Data is kept raw (unscaled) through the entire workflow
- A Pipeline([preprocessor, model]) is built automatically when you train
- cross_validate() refits the preprocessor inside each fold using only the training data of that fold
- Your validation fold never influences the scaler statistics

What this means for you:

- CV scores are now scientifically valid — no hidden leakage
- Saved models are complete Pipelines that accept raw DataFrames for inference
- All preprocessing happens automatically and correctly behind the scenes
- The 20% internal test split and external validation metrics were already correct and are unchanged

External Validation

When you load an external validation dataset:

- Models report metrics on BOTH internal test split AND external validation set
- Three new columns in Results: "Val. RMSE/Acc", "Val. MAE", "Val. R2/F1"
- No data leakage — external data is NEVER used for training

Getting Started

Launching the Application

21. Double-click the AutoAIM Studio.exe file
22. The main window will open with multiple tabs at the top
23. Start by loading your data in the Data tab

Workflow Overview

The typical workflow follows these steps:

24. Load Data → Data Tab
25. Link Crystal Structures → Data Tab (optional, v2.0 — connect CIF files)
26. Health Check → Data Tab (optional, v2.0 — analyze data quality)
27. Preprocess → Data Tab (handle missing values, encode categoricals)
28. Generate Features → Data Tab (optional, composition descriptors or crystal features)
29. Train Models → Training Tab or Neural Network Tab
30. Optimize → Optimization Tab (optional, for hyperparameter tuning)
31. Create Ensembles → Ensemble Tab (optional, combine models)
32. Analyze Results → Results Tab
33. Export Session → Results Tab (v2.0 — save complete report)
34. Interpret Models → Explainability Tab
35. Make Predictions → Predict Tab (v2.0 — with confidence intervals)

Data Tab

The Data Tab is your starting point for loading and preparing your dataset.

Loading Data

36. Click "Load Data" button
37. Select your data file (CSV or Excel format supported)
38. The data preview will appear in the table below

Supported Formats:

- CSV files (.csv)
- Excel files (.xlsx, .xls)

Data Preview

After loading, you'll see:

- Shape: Number of rows and columns
- Columns: List of all column names
- Data Types: Automatic detection of numerical and categorical columns
- Sample Data: First few rows of your dataset

Target Column Selection

39. Select your target variable from the "Target Column" dropdown
40. The system will automatically detect if it's a regression or classification problem

Link Crystal Structures (v2.0)

Connect crystal structure files (CIF) to your tabular dataset.

Prerequisites:

- Your CSV must have an ID column that matches your CIF filenames
- CIF files must be named {ID_value}.cif (e.g., if ID is mp-1234, the file must be mp-1234.cif)

Steps:

41. Click "Link Crystal Structures" button
42. AutoAIM shows ID column candidates (columns with mostly unique values)
43. Select your ID column from the list
44. Choose the folder containing your CIF files
45. AutoAIM reports: matched CIFs, missing CIFs, and duplicate IDs
46. Click Yes to proceed

After linking:

- 45 crystal features are added (11 structural + 34 periodic compositional including oxidation states)
- The Magpie "Generate composition features" checkbox is disabled (composition already included)
- The ID column is automatically excluded from training features

Link Molecular Structures (v2.0)

Connect molecular structure files (.xyz) to your tabular dataset.

What is a molecule vs. a crystal?

- Molecules are individual, isolated structures (e.g., H_2O, caffeine, a protein ligand) — they have no periodic boundary, no unit cell
- Crystals are periodic structures with a unit cell that repeats in 3D space (e.g., SiO_2 quartz, Fe_2O_3 hematite)
- If your .xyz file has a unit cell, use Link Crystal Structures instead

Prerequisites:

- Your CSV must have an ID column that matches your .xyz filenames (without extension)
- .xyz files must be named {ID_value}.xyz

Steps:

47. Click "Link Molecular Structures" button
48. Select your ID column (the one matching .xyz filenames)
49. Choose the folder containing your .xyz files
50. AutoAIM reports: matched files, missing files
51. Click Yes to proceed

After linking:

- 149 molecular features are added (composition + geometry 3D + topology + 34 periodic properties + oxidation states)
- The ID column is automatically excluded from training features

Alternative: Direct training on .xyz files

- If you only have .xyz files and no CSV, click "Load Molecular Structures" or "Load Molecular Directory"
- The extracted molecular features become your full dataset
- Select a target column (if available) and train directly

Data Health Check (v2.0)

Analyze your dataset quality before training.

Steps:

52. Click "Data Health Check" button
53. Review the quality score (0-100) and severity level
54. Check detected issues and recommendations
55. Fix issues if needed before training

Checks performed:

- Missing values (severity based on percentage)
- Duplicate rows
- Constant columns
- Outliers (IQR method)
- Scaling mismatches between features
- High correlations between features (>90%)
- Sample size adequacy
- Class imbalance (for classification)
- Target column quality

Preprocessing Options

Handle Missing Values

- Checkbox: "Handle missing values"
- When enabled, missing numerical values are imputed with the median
- Missing categorical values are imputed with the most frequent value

Encode Categorical Variables

- Checkbox: "Encode categorical variables"
- When enabled, categorical variables are one-hot encoded
- This expands categorical columns into multiple binary columns

Scale Features

- Checkbox: "Scale features"
- When enabled, features are standardized (zero mean, unit variance)

- Recommended for algorithms sensitive to feature scales (SVR, Neural Networks)

Generate Composition Features

- Checkbox: "Generate composition features (106 descriptors)"
- Requires: A column containing chemical formulas (e.g., "Fe2O3", "SiO2")
- When enabled, the system extracts 106 composition-based descriptors including:
- Atomic number statistics (mean, std, min, max)
- Atomic mass statistics
- Electronegativity statistics
- Covalent radius statistics
- Ionization energy statistics
- And many more material properties

Note: The composition feature generator is a custom implementation designed for materials science applications. It analyzes chemical formulas and computes aggregate statistics based on elemental properties.

Preparing Data

After configuring options:

56. Click "Prepare Data" button
57. The system will split data into train/test sets (80/20 by default)
58. All selected preprocessing steps are applied
59. A confirmation message will appear when complete

Training Tab

The Training Tab allows you to train multiple machine learning models simultaneously.

Model Selection

Select one or more models from the list:

- Random Forest: Ensemble of decision trees, good for general-purpose prediction
- Gradient Boosting: Sequential ensemble, often achieves high accuracy (sequential — slower on large datasets)
- Histogram Gradient Boosting: v2.0 — Parallel gradient boosting via histograms. 10-100x faster than classic GB on large datasets. Recommended for gradient boosting workloads.
- XGBoost: Optimized gradient boosting, industry standard
- LightGBM: Fast gradient boosting with leaf-wise growth
- CatBoost: Gradient boosting optimized for categorical features
- SVR: Support Vector Regression, good for smaller datasets
- Ridge: Linear regression with L2 regularization, simple baseline

- Gaussian Process (v2.1): Probabilistic regressor with native uncertainty quantification (±1σ posterior standard deviation) and full kernel family tuning (RBF, Matern, RationalQuadratic, WhiteKernel, RBF+WhiteKernel). Best for small-to-medium datasets where prediction confidence matters. Kernel hyperparameters are fully optimizable via Bayesian optimization.

Training Configuration

- Problem Type: Automatically detected (regression/classification)
- CV Strategy: How cross-validation splits your data (default: K-Fold). See Choosing a CV Strategy below. Options: K-Fold, Stratified K-Fold (quantiles), Group K-Fold, Leave-One-Out (LOO), Leave-One-Group-Out (LOGO).
- Cross-Validation Folds: Number of CV folds (default: 5). Disabled for LOO/LOGO (folds are set automatically).
- Random State: Seed for reproducibility (default: 42). Controls data split + model init + CV folds. See Random State & Reproducibility below.

Training Process

60. Select desired models (check the boxes)
61. Click "Train Selected Models"
62. Progress will be shown in the status bar
63. Results will appear in the Results Tab automatically

Manual (Custom) Parameter Training

For advanced users who want full control over model hyperparameters:

64. Click on a model name in the list (not the checkbox)
65. The Manual Parameter Dialog opens with all available parameters
66. Set your custom values (e.g., n_estimators, max_depth)
67. Configure common parameters:

- random_state: Only shown if the model accepts it (hidden for deterministic models like SVR, Ridge)
- n_jobs: Only shown if the model supports parallel training (hidden for single-threaded models)
- cv_folds: Configurable cross-validation folds (v2.0 — was hardcoded to 5). Disabled when LOO/LOGO is selected.
- cv_strategy: Full cross-validation configuration via the shared CV widget (v2.0) — K-Fold, Stratified K-Fold (quantiles), Group K-Fold, LOO, or LOGO, with bins / group source / clusters / group column shown conditionally. See Choosing a CV Strategy below.

68. Click Train

v2.0 Note: Deterministic models (SVR, Ridge) don't show random_state or n_jobs in the dialog since they don't accept these parameters. This prevents confusion when setting values that have no effect.

v2.0 Note: When LOO-CV or LOGO is selected (in the Training tab or the Manual Parameter Dialog), the CV Folds spinner is automatically disabled and an info message reminds you that the folds are set automatically (N samples for LOO, N groups for LOGO).

Choosing a CV Strategy (v2.0)

Cross-validation estimates how well a model generalizes. The right strategy depends on your dataset size, whether your target is skewed, and whether your samples form groups that could "leak" information between train and test. AutoAIM supports five strategies, available consistently across sklearn regressors, Neural Networks, and Ensembles; the same controls appear in the Training tab, the Optimization tab, the Neural Network tab, the Ensemble tab, and the Manual Parameter dialog. (Quantile stratification and the reserved bin are regression-only; classification folds are stratified by class automatically.)

Your situation	Recommended strategy	Why
General case, N > 30, roughly balanced target	K-Fold	Standard, fast, robust
Skewed target (e.g. most Tc low, a few high)	Stratified K-Fold (quantiles)	Every fold preserves the target distribution
Samples form chemical families / batches / synthesis groups	Group K-Fold	A group never crosses train/test → no leakage
Very small dataset (N ≤ 30)	Leave-One-Out (LOO)	Maximizes training data per fold
Few groups, want strict group held-out test	Leave-One-Group-Out (LOGO)	Each group tested once; folds = n_groups

The five strategies:

69. K-Fold (default). Random folds. For classification, folds are automatically stratified by class.
70. Stratified K-Fold (quantiles) — regression only. The target is binned into quantiles and each fold preserves the target distribution. Use it for skewed targets. Not available for classification (which is already class-stratified under K-Fold).

How the Quantile Bins control works: When you select this strategy, a Quantile Bins spinner appears (only for this strategy), with range 2–20 and default 5. The target is split into that many quantiles (equal-count bins — with 5 bins, each holds ~20% of the samples sorted by target

value). Each bin becomes a pseudo-class, and StratifiedKFold then gives every fold a proportional share of every bin, so no fold ends up with only low or only high values.

The bin count is a request, not a guarantee. AutoAIM enforces a hard rule — each bin must contain at least as many samples as there are folds (otherwise a fold could miss a bin entirely). So the effective maximum is N ÷ folds (with N=50 and 5 folds, at most 10 bins). If your target has many repeated values or the request exceeds what the data supports, the bin count is reduced automatically and an orange info note tells you exactly what was used (e.g. "Quantile bins reduced from 10 to 5 …"). If the target is constant, or not even a 2-bin median split is possible, the run is blocked with a message suggesting standard K-Fold. The session report header shows the requested bins and any reduction as a warning line.

Reserve a bin for an exact value (v2.0). Many materials targets have a degenerate value meaning "property absent": Tc = 0 K (non-superconductor), Eg = 0 eV (metal), E_hull = 0 (stable phase), activity = 0 (inactive catalyst). These are point masses, not continuous values, and if they land unevenly across folds the per-fold scores become hard to compare. Enabling "Reserve a bin for an exact value" (visible only for this strategy) isolates every sample matching that value into its own stratum, so every fold carries the same proportion of it; the remaining samples are binned into the other bins − 1 quantiles.

- Special Value: the exact target value to isolate (e.g. 0.0).
- Tolerance: how close a value must be to count (default 0.0 = exact match). Increase it only if your data stores the value as "below detection threshold" (e.g. 0.01 instead of 0).
- Auto-detection: if AutoAIM detects a dominant repeated value in your target (at least ~5% of samples and at least 2 per fold), a cyan suggestion appears — e.g. "Detected 25 samples with target = 0 (28%). This usually means 'property absent'. Consider enabling 'Reserve a bin…'". The suggestion is guidance only; nothing is applied until you enable the checkbox.
- Manual use is validated with real numbers, not a generic warning. If your chosen value matches too few samples to stratify (fewer than the number of folds), the run is blocked. If it is the majority (>50%) or a very small fraction (<5%), a warning shows the actual count and percentage and asks you to confirm — so you can reconsider whether the reserved bin is meaningful for your data.

Note: a reserved bin improves the honesty of the CV estimate; it does not change the underlying modeling. For a heavily zero-inflated target, the more rigorous approach is a two-stage model (classify "does it exhibit the property?" then regress "by how much"), which is outside the current scope.

71. Group K-Fold. You define groups; samples from the same group never appear on both sides of a train/test split. Choose the grouping source:

- Chemical system — same set of elements (all Y-Ba-Cu-O together). The standard anti-leakage split for materials data.
- Exact formula — same stoichiometry together (reduced form, so $Y_2Ba_4Cu_6O_{14}$ groups with $YBa_2Cu_3O_7$).

- Clustering (advanced) — agglomerative clustering over element fractions (deterministic; set the cluster count).
- Dataset column — any column: synthesis batch, laboratory, source, year.

72. Leave-One-Out (LOO). Each sample left out once; folds = N. Blocked for N > 100; warns for 30 < N ≤ 100.
73. Leave-One-Group-Out (LOGO). Each group left out once; folds = number of groups. Needs ≥ 3 groups; blocked for > 100 groups; warns for 30 < n_groups ≤ 100.

Metrics note: For LOO and LOGO (and any grouped fold that holds a single sample), R^2 is mathematically undefined for a 1-sample test fold. AutoAIM computes a global R^2 over all out-of-fold predictions and reports per-sample / per-group MAE. For K-Fold, Stratified, and Group K-Fold it reports per-fold Train Score + Test Score (plus a Test Size column when folds are uneven, as in Group K-Fold).

Warnings and blocks: AutoAIM validates your choice before running and will block impossible combinations (e.g. more folds than groups, LOGO with < 3 groups, a group column with missing values) with a clear message, and warn (asking for confirmation) about expensive or risky ones (LOO/LOGO cost, highly imbalanced groups).

Understanding Results

After training, each model shows:

- RMSE: Root Mean Squared Error (lower is better)
- MAE: Mean Absolute Error (lower is better)
- R^2: Coefficient of determination (higher is better, max 1.0)
- Training Time: How long the training took

Details Popup — Cross-Validation Scores

Click "Details" on any trained model to see the full cross-validation breakdown:

K-Fold CV (default):

```
Fold |  Train Score |   Test Score
-----+--------------+-------------
   1 |     0.999960 |     0.975628
   2 |     0.999960 |     0.973907
 ...
Mean |     0.999962 |     0.949832
 Std |     0.000003 |     0.050600
```

Group K-Fold (adds a Test Size column because grouped folds are uneven):

```
Fold |  Train Score |   Test Score | Test Size
-----+--------------+--------------+----------
   1 |     0.999960 |     0.975628 |        12
```

```
   2 |     0.999960 |     0.973907 |         8
 ...
Mean |     0.999962 |     0.949832 |
 Std |     0.000003 |     0.050600 |
```

Leave-One-Out (LOO-CV):

```
R² (global): 0.960446

Per-sample Mean Absolute Error (MAE):
Sample  1: 0.019916
Sample  2: 0.003685
 ...
Mean MAE ± Std: 0.012872 ± 0.008515
```

Leave-One-Group-Out (LOGO):

```
R² (global): 0.978000

Per-group Mean Absolute Error (MAE):
Group  1: 0.120000
Group  2: 0.110000
 ...
```

Important: For LOO-CV and LOGO, R^2 per fold is mathematically undefined (each fold has a single test sample or group). AutoAIM computes a global R^2 over all out-of-fold predictions and reports per-sample / per-group MAE. This is the correct scientific approach for these strategies.

Random State & Reproducibility (v2.0)

AutoAIM lets you control the random seed everywhere randomness appears, so results are fully reproducible. When you set a Random State / Random Seed, AutoAIM applies it to all of the following at once:

- Train/test split (Training tab "Random State")
- Model weight initialization / bootstrap sampling (stochastic models)
- CV fold shuffling (the fold assignment follows your seed)
- Optimizer search sequence (Bayesian sampler seed; Grid Search is deterministic)

Where to set it

Tab	Control	What it controls
Training	Random State	data split + model init + CV folds
Optimization	Random Seed	model init + search sampler +

		CV folds
Neural Network	Random Seed (training) / Random Seed (optimization)	weight init + data shuffling + CV folds
Ensemble	Random Seed (training) / Random Seed (weight opt)	ensemble CV folds + weight search
Manual Parameter dialog	random_state	model init + CV folds (shown only for models that accept it)

Which models use it

Stochastic models honor the seed: Random Forest, Gradient Boosting, Histogram Gradient Boosting, XGBoost, LightGBM, CatBoost, Neural Networks, Ensembles. Deterministic models (SVR, Ridge, k-NN) do not — they have no randomness to seed, so the control is disabled/hidden for them and setting a value has no effect on the model itself (it still affects the CV split shuffling).

Notes

- CatBoost uses random_seed internally; AutoAIM maps your Random State to it automatically (this was a silent bug fixed in v2.0 — earlier the optimizer passed random_state, which CatBoost ignores).
- Same seed + same data + same settings → identical results, guaranteed. This is covered by automated reproducibility tests (test_random_state.py).

Gaussian Process (v2.0 → v2.1)

Gaussian Process (GP) is a probabilistic machine learning model that provides predictions together with a native uncertainty estimate — the posterior standard deviation. This makes it uniquely valuable for materials science, where knowing "how confident is the model?" is often as important as the prediction itself.

v2.1: Kernel Family Tuning

AutoAIM Studio v2.0.0 introduces full kernel family support — the most scientifically complete GP implementation in the AutoML ecosystem for materials science.

Supported Kernel Families

Kernel	Description	Best For
RBF	Infinitely differentiable,	General-purpose, well-

	smooth functions	behaved data
Matern	Controlled smoothness via nu (0.5, 1.5, 2.5)	Data with known roughness; 0.5 = once differentiable (exponential), 2.5 = smooth (almost RBF)
RationalQuadratic	Mixture of RBFs with different length-scales	Multi-scale data, varying correlation ranges
WhiteKernel	Independent observation noise	Explicit noise modeling, heteroscedastic data
RBF + WhiteKernel	Signal + noise decomposition	Separating signal covariance from noise covariance

Tuneable Hyperparameters

Parameter	Type	Range	Description
kernel_type	categorical	RBF, Matern, RQ, White, RBF+White	Kernel family selector
kernel_length_scale	float_log	1e-3 → 100.0	Characteristic length scale (most important GP hyperparameter)
kernel_nu	categorical	0.5, 1.5, 2.5	Matern smoothness (lower = rougher)
kernel_alpha_rq	float_log	1e-3 → 100.0	Rational Quadratic tail heaviness
kernel_noise_level	float_log	1e-5 → 1.0	WhiteKernel noise magnitude
alpha	float_log	1e-10 → 1.0	Tikhonov regularization (observation noise)
n_restarts_optimizer	int	1 → 20	L-BFGS-B restarts for marginal likelihood
normalize_y	categorical	True, False	Normalize target before fitting

Bayesian optimization (Optuna TPE) is recommended for kernel tuning — it can optimize all 8 parameters simultaneously. Grid Search supports scalar GP params (alpha, n_restarts_optimizer, normalize_y) but automatically excludes kernel parameters with a warning.

Why use Gaussian Process?

Situation	Why GP excels
Small-to-medium datasets (N < 2,000)	GP interpolates well; uncertainty is largest far from training data
You need prediction confidence	Every prediction comes with ±1σ (standard deviation of the posterior)
Exploring a new chemical space	High uncertainty flags regions where the model has little information
Benchmarking / baseline	Simple, theoretically grounded alternative to tree ensembles

Scalability warning

GP training scales as $O(n^3)$ due to matrix inversion. For datasets above ~2,000 samples, training becomes slow. AutoAIM shows a console warning when GP is selected with N > 2,000. For very large datasets, use a tree-based model instead.

What you get

- Prediction: The mean of the posterior distribution (same as any regressor)
- prediction_std: The standard deviation of the posterior (unique to GP). Lower = more confident.
- Kernel: Configurable kernel family (RBF, Matern, RQ, White, RBF+White) with automatic hyperparameter optimization
- Alpha: Learned noise level (how much the data deviates from a perfect smooth function)

Where it appears

- Training Tab: Select "Gaussian Process" from the model list and train normally. Use the "Custom Parameters" button to select a kernel family and tune kernel hyperparameters manually.
- Optimization Tab: Optimize all GP parameters including kernel_type, kernel_length_scale, kernel_nu, kernel_alpha_rq, kernel_noise_level, alpha, n_restarts_optimizer, and normalize_y. Bayesian optimization is recommended for kernel tuning.
- Predict Tab: Predictions include an extra prediction_std column with ±1σ uncertainty

- Model Card: Shows kernel family, fitted kernel string, length scale, nu (Matern), alpha (RQ), noise level, and log marginal likelihood
- Session Export: GP Metadata section includes full kernel provenance for scientific reproducibility

Interpreting prediction_std

prediction_std	Interpretation
< 0.1	High confidence — prediction is near training data
0.1 – 0.5	Moderate confidence — some extrapolation
> 0.5	Low confidence — far from training data or high noise

Important: prediction_std is the model's own epistemic uncertainty, not a confidence interval. For a 95% confidence interval, multiply by ~1.96 (assuming approximately Gaussian posterior).

Parameters

Parameter	Default	Description
alpha	1e-10	Noise level added to diagonal of kernel matrix
n_restarts_optimizer	10	Number of random restarts for L-BFGS-B kernel optimization
normalize_y	True	Normalize target to zero mean, unit variance before fitting
random_state	42	Seed for optimizer restarts (affects initial points)
kernel_type	RBF	Kernel family: RBF, Matern, RationalQuadratic, WhiteKernel, RBF+WhiteKernel
kernel_length_scale	1.0	Characteristic length scale of the kernel (smaller = wigglier, larger = smoother)
kernel_nu	2.5	Matern smoothness: 0.5

		(rough), 1.5, 2.5 (smooth, almost RBF)
kernel_alpha_rq	1.0	Rational Quadratic tail heaviness (low = heavy tails, high ≈ RBF)
kernel_noise_level	1e-5	WhiteKernel noise magnitude (explicit observation noise)

Model Provenance

v2.1: For scientific reproducibility, the Results Tab Details popup and Session Export capture the full kernel provenance:

- kernel_family: RBF, Matern, RationalQuadratic, WhiteKernel, or Composite
- kernel_length_scale: Fitted length scale (4 decimal precision)
- kernel_nu: Fitted Matern smoothness (if applicable)
- kernel_alpha: Fitted RQ tail parameter (if applicable)
- kernel_noise_level: Fitted WhiteKernel noise (if applicable)
- log_marginal_likelihood_value: Optimized marginal log-likelihood (model evidence)

This metadata is essential for publication-ready model cards and ensures that any GP model can be fully reconstructed from the session export.

SHAP support

GP uses KernelExplainer (fallback) because it is not a tree, linear, or neural model. This is slower than TreeExplainer. For datasets > 500 samples, SHAP values may take noticeable time.

Neural Network Tab

The Neural Network Tab provides a visual interface for designing and training custom neural networks.

v2.0 Layout: The tab is split into two panels. The left panel (architecture configuration, ~75% of screen width) contains all controls: layer settings, training configuration, a Cross-Validation section, hyperparameter optimization, and action buttons (Auto-Optimize, Apply Best, Train NN). The left panel is vertically scrollable, so every control and button stays reachable on typical laptop and desktop screens — nothing is ever cut off. The right panel (~25% of screen width) shows Training Progress, Architecture Preview, Training Results, and Model Information consoles.

Architecture Design

Number of Hidden Layers

- Use the spinbox to set 1-5 hidden layers
- Each layer can be configured independently

Layer Configuration

For each hidden layer, you can set:

- Units: Number of neurons (16-2048)
- Activation Function: ReLU, Leaky ReLU, Tanh, Sigmoid, GELU, ELU
- Dropout Rate: Regularization (0.0-0.5)
- Batch Normalization: Enable/disable batch normalization

Architecture Preview

Click "Update Architecture Preview" to see:

- Input layer dimensions
- Hidden layer configurations
- Output layer dimensions
- Total number of parameters
- Trainable parameters

Training Configuration

- Epochs: Number of training iterations (10-10,000)
- Batch Size: Samples per gradient update (8-512)
- Learning Rate: Step size for optimization (0.00001-0.1)
- Optimizer: Adam, AdamW, SGD, RMSprop
- Weight Decay (L2): Regularization strength (0.0-0.1)
- Early Stopping Patience: Stop if no improvement after N epochs (5-500)

Cross-Validation (v2.0)

Neural Networks now support the same five CV strategies as sklearn regressors (see Choosing a CV Strategy in the Training Tab): K-Fold, Stratified K-Fold (quantiles, with optional reserved bin), Group K-Fold, LOO, and LOGO. Each CV fold re-trains a fresh copy of the network, so the estimate reflects true generalization. Note that LOO/LOGO train one network per sample/group — expensive for NNs, so use them only on small datasets; the usual blocks/warnings apply.

Training Process

74. Configure architecture and training parameters
75. Click "Train Neural Network"

76. Watch training progress:

- Progress bar shows current epoch
- Log shows train/validation loss per epoch

77. Training stops automatically when:

- Maximum epochs reached, OR
- Early stopping triggered (no improvement)

NN Architecture Optimization

v2.0: You can optimize your neural network architecture using Bayesian optimization:

78. Configure initial architecture (this will be the starting point)
79. Click "Optimize Architecture"
80. Set optimization parameters:

- Number of Trials: How many architectures to try
- Max Layers: Maximum number of hidden layers
- Min/Max Units: Range for neurons per layer
- CV Folds: Cross-validation folds for evaluation

81. Click "Start Optimization"
82. After optimization completes, the best architecture is automatically applied
83. Click "Train with Best" to train with the optimized architecture

v2.0 Note: Optimized NNs show [OPTIMIZATION DETAILS] in the session export, including the number of trials, best score, and optimization configuration — same as sklearn optimized models.

Saving Neural Networks

After training:

84. Click "Save Model"
85. Choose format:

- PyTorch Model (.pt): Simple format
- Model Bundle: Includes manifest.json, model, and preprocessor (recommended for Predict tab)

Optimization Tab

The Optimization Tab finds the best hyperparameters for your models using either Bayesian optimization (default) or Grid Search (exhaustive evaluation).

Search Strategy Selection

Bayesian (Optuna) — Recommended for most cases:

- Smart exploration: tests the most promising combinations first
- Needs fewer trials (50–100) for excellent results
- Supports pruning (early stopping of bad trials)
- Handles continuous and discrete parameters natively
- Best for 3+ parameters or wide ranges

Grid Search — Deterministic and reproducible:

- Evaluates every combination in the parameter grid
- Same inputs always produce the same result
- Best for ≤3 parameters with small discrete ranges
- Useful as a baseline comparison against Bayesian results
- Choose resolution: Coarse (3 points), Medium (5), or Fine (7) per parameter

AutoAIM shows an estimated combination count before starting Grid Search. If the estimate exceeds 5,000, Grid Search is blocked with a warning.

v2.0 UX: When Bayesian optimization is selected, the Grid Resolution selector is automatically disabled and an orange info label reminds you that "Grid Resolution is only used with Grid Search. Bayesian optimization uses adaptive sampling, not fixed grids."

Cross-Validation Strategy

The Optimization tab offers the same five CV strategies as the Training tab (see Choosing a CV Strategy in the Training Tab section): K-Fold, Stratified K-Fold (quantiles), Group K-Fold, LOO, and LOGO. The selected strategy applies to every trial of both Bayesian and Grid Search optimization — each candidate hyperparameter set is scored with that CV. Grouped strategies (Group K-Fold / LOGO) keep group membership intact inside the optimizer, so hyperparameters are selected without chemical/batch leakage.

For LOO and LOGO, optimization uses Mean Absolute Error as the objective (R^2 is undefined for single-sample folds); for the other strategies it uses R^2.

The same blocks and warnings apply (e.g. LOO blocked for N > 100, more folds than groups blocked, LOGO blocked for > 100 groups).

Model Selection

Model Selection

Select a single model to optimize from the dropdown:

- Random Forest
- Gradient Boosting
- XGBoost

- LightGBM
- CatBoost
- SVR
- Gaussian Process (v2.1 — full kernel tuning)

Optimization Settings

- Search Strategy: Bayesian (Optuna) or Grid Search
- Grid Resolution (Grid Search only): Coarse (3 pts) / Medium (5) / Fine (7) per parameter. Disabled when Bayesian is selected.
- Number of Trials (Bayesian only): How many hyperparameter combinations to try (10–200)
- CV Strategy: K-Fold, Stratified K-Fold (quantiles), Group K-Fold, LOO, or LOGO (see Choosing a CV Strategy)
- CV Folds: Cross-validation folds for evaluating each combination. Disabled for LOO/LOGO (folds set automatically).
- Group By / Bins / Clusters / Group Column: shown only for grouped or stratified strategies
- Random Seed: For reproducible optimization (model init + search sampler + CV folds). Disabled for deterministic models (SVR, k-NN), which accept no seed.
- Timeout: Maximum optimization time in seconds (0 = no limit)
- Enable Pruning (Bayesian only): Stop unpromising trials early (recommended)

Custom Parameter Ranges

Click "Edit Parameter Ranges" to customize the search space:

For each parameter, you can set:

- Min Value: Lower bound
- Max Value: Upper bound
- Type: Integer or Float

Available Parameters by Model:

Model	Parameters
Random Forest	n_estimators, max_depth, min_samples_split, min_samples_leaf
Gradient Boosting	n_estimators, max_depth, learning_rate, subsample
Histogram Gradient Boosting	max_iter, max_depth, learning_rate, min_samples_leaf, max_bins, l2_regularization
XGBoost	n_estimators, max_depth, learning_rate, subsample, colsample_bytree, reg_alpha,

	reg_lambda
LightGBM	n_estimators, max_depth, learning_rate, num_leaves, subsample, colsample_bytree, reg_alpha, reg_lambda
CatBoost	iterations, depth, learning_rate, l2_leaf_reg
SVR	C, epsilon, gamma
Gaussian Process	kernel_type, kernel_length_scale, kernel_nu, kernel_alpha_rq, kernel_noise_level, alpha, n_restarts_optimizer, normalize_y (v2.1)

Click "Reset to Default" to restore original ranges.

Optimization Process

86. Select model and configure settings
87. (Optional) Edit parameter ranges
88. Click "Start Optimization"
89. Monitor progress:

- Progress bar shows current trial
- Log shows trial results in real-time

Results

After optimization:

- Best Parameters: Optimal hyperparameter values found
- Best Score: Best performance achieved
- Number of Trials: Total trials completed
- Optimization Time: Total time elapsed

v2.0: Click "Details" to see the per-trial breakdown, or "Train with Best" to train a final model. The success message and Details panel now show Train Score + Test Score side by side for K-Fold, and R^2 (global) + per-sample MAE for LOO-CV.

Train with Best Parameters

90. After optimization completes, click "Train with Best"
91. The model is trained using the optimal hyperparameters found
92. The trained model appears in the Results Tab

v2.0 Note: Multiple optimizations of the same algorithm are now preserved. If you optimize XGBoost with 5 CV folds and then again with 3 folds, both optimizations remain available. Each trained model links to its correct optimization configuration in the session export.

- Trial History Table: Complete history of all trials

Saving Optimized Models

Click "Save Optimized Model":

93. Choose format:

- Model Bundle: Includes manifest.json (recommended)
- Legacy Format: Joblib/pickle only

94. Select destination folder
95. Model is saved and appears in Results Tab

Results Tab

The Results Tab displays all trained, optimized, and ensemble models.

Model List

All models are shown in a table with:

- Model Name: Identifier
- Type: Trained, Optimized, Neural Network, or Ensemble
- RMSE: Root Mean Squared Error
- MAE: Mean Absolute Error
- R^2: Coefficient of determination

Actions

For each model, you can:

- Details: View detailed information about the model
- Delete: Remove the model from the list
- Export: Save as model bundle or legacy format

Model Details

Clicking "Details" shows:

- Model type and source
- Training/optimization time
- All performance metrics
- Best parameters (for optimized models)
- Feature importance (if available)
- Neural network architecture (for NN models)
- Cross-Validation breakdown (v2.0):
- K-Fold: Train Score + Test Score per fold, with separate Mean and Std rows

- LOO-CV: R^2 (global) + per-sample MAE table, with Mean MAE ± Std
- v2.1: Gaussian Process kernel provenance (for GP models):
- Kernel family (RBF, Matern, RationalQuadratic, WhiteKernel, Composite)
- Fitted kernel string, length scale, nu, alpha, noise level
- Log marginal likelihood (model evidence)
- Prediction uncertainty statistics (mean, std, min, max of prediction_std)

Learning Curves

Select a model and click "Generate Learning Curves" to visualize:

- Training vs validation loss over epochs
- Helps diagnose overfitting/underfitting

Exporting Models

96. Select model from dropdown
97. Click "Export Model Bundle"
98. Choose destination folder
99. Bundle contains:

- manifest.json: Model metadata
- pipeline.joblib or model.pt: Trained model
- preprocessor.joblib: Preprocessing pipeline (for NN)

Session Export (v2.0 → v2.1)

Export your complete session with all results:

100. Click "Export Session" button
101. Choose format:

- .txt: Human-readable report (recommended)
- .md: Markdown for documentation
- .csv: Spreadsheet format
- .json: Machine-readable

102. The export includes:

- All trained models with metrics and per-fold CV scores
- All optimized models with [OPTIMIZATION DETAILS] (trials, config, best params)
- All neural networks with architecture, training history, and CV fold breakdown
- All optimized NNs with optimization time and details
- Ensemble results
- Final ranking table
- Reproducibility checklist (random seeds, configs)
- v2.1: Clean model parameters (no Pipeline noise, no model__ prefix)

- v2.1: GP Metadata section with kernel family, fitted hyperparameters, and prediction uncertainty statistics

v2.1 Cleanup: The export now filters out internal Pipeline noise (preprocessor__*, steps, memory, verbose) and strips the model__ prefix from parameter keys. This makes the report significantly cleaner and more readable for both beginners and advanced users.

v2.0 Reliability: The export uses per-result error handling — if one model fails to export, the others still appear. Previously, a single bad result could destroy the entire export.

v2.0 Accuracy: Optimization configurations are read from the stored result (captured at optimization time), not from the current GUI values. This means if you optimized XGBoost with 5 folds, changed to 3 folds, and optimized again, both exports show their correct configurations.

v2.0 NN Optimization Details: Optimized neural networks (e.g., NN_Optimized) now show [OPTIMIZATION DETAILS] inline, including trials, best score, and optimization configuration — same format as sklearn optimized models.

v2.0 CV Report Format: The exported CV table shows Train Score and Test Score in separate columns, with Mean and Std on separate rows. For LOO-CV, the report shows R^2 (global) at the top and a per-sample MAE table below. Example:

```
Cross-Validation (5-Fold):
    Fold |  Train Score |   Test Score
  -------+--------------+-------------
       1 |     0.999960 |     0.975628
       2 |     0.999960 |     0.973907
     ...
    Mean |     0.999962 |     0.949832
     Std |     0.000003 |     0.050600
```

Ensemble Tab

The Ensemble Tab allows you to combine multiple models for improved predictions.

Selecting Models

103. Click "Refresh Model List" to load available models
104. Select multiple models from the list (Ctrl+Click for multiple)
105. Available sources:

- Trained models from Training Tab
- Optimized models from Optimization Tab

Ensemble Configuration

- Ensemble Type:

- Averaging: Mean of all model predictions
- Voting: Weighted voting (for classification)
- Weighted: Weighted average (equal weights by default)
- Number of Models: Shows how many models are selected

Cross-Validation (v2.0)

Ensembles now support the same five CV strategies as sklearn regressors (see Choosing a CV Strategy): K-Fold, Stratified K-Fold (quantiles, with optional reserved bin), Group K-Fold, LOO, and LOGO. The chosen strategy is used both when training the ensemble and when optimizing the ensemble weights (weight optimization shares the strategy/groups but uses its own fold count). Grouped strategies keep groups intact, so the ensemble's reported score is free of chemical/batch leakage. Ensembles are most reliable with N ≥ 100.

Creating Ensemble

106. Select 2 or more models
107. Choose ensemble type
108. Click "Create Ensemble"
109. Wait for evaluation to complete

Ensemble Results

After creation, you'll see:

- Ensemble Name: Auto-generated identifier
- Test Metrics: RMSE, MAE, R^2 on test set
- Training Metrics: RMSE, MAE, R^2 on training set

Saving Ensembles

Click "Save Ensemble Model":

110. Choose format (Bundle or Legacy)
111. Select destination folder
112. Ensemble appears in Results Tab and can be used in Explainability

Explainability Tab

The Explainability Tab provides model interpretation tools including SHAP values, permutation importance, and partial dependence plots.

Model Selection

113. Click "Refresh Model List" to load available models
114. Select a model from the dropdown
115. Available models: Trained, Optimized, Neural Network, Ensemble

v2.0 FIX: Neural Networks no longer appear duplicated. Each NN appears once as "Neural Network (Neural Network)".

Analysis Options

Select which analyses to run:

- Compute SHAP Values: Feature contributions using SHAP
- Compute Permutation Importance: Feature importance via permutation
- Compute Partial Dependence Plots: Feature effect visualization
- Analyze Domain Applicability: Data coverage analysis

Number of Features

Set how many top features to display (1-100, default: 10)

Running Analysis

116. Select model and options
117. Click "Run Explainability Analysis"
118. Monitor progress in the log area
119. Results appear in tabs below

v2.0 NOTE: The log area now shows detailed diagnostics for each importance type (sum, all_zero, all_nan) to help verify results.

Results Tabs

Feature Importance

Table showing:

- Feature: Feature name
- Model Importance: Built-in importance (if available) or permutation-based surrogate
- Permutation: Permutation importance score
- SHAP: Mean absolute SHAP value
- Average Rank: Combined ranking across methods

Notes:

- For Neural Networks, Model Importance shows "N/A" as they don't have built-in feature importance
- For HistGradientBoosting and LightGBM: Model Importance is computed via a fast permutation-based surrogate (v2.0). Previously these showed zeros due to oversubscription.
- For SVR and Ridge: Model Importance comes from coefficients or permutation; random_state/n_jobs are hidden from ManualParamDialog as these models are deterministic

SHAP Values

Summary of SHAP analysis:

- Global feature importance (top N features)
- Mean absolute SHAP value per feature

v2.0 SHAP Explainer Selection (automatic, no user action needed):

- TreeExplainer for tree models (Random Forest, XGBoost, LightGBM, Histogram Gradient Boosting, Gradient Boosting, CatBoost) — exact and fast
- LinearExplainer for linear models (Ridge, Logistic Regression) — exact
- DeepExplainer for Neural Networks — specialized for PyTorch
- KernelExplainer only as fallback for generic models (SVR, k-NN, Gaussian Process) — slower, approximate

v2.0 Performance: Explainability uses smart parallelization — single-threaded models (SVR, Ridge) use all CPU cores for permutation importance, while multi-threaded models (RF, XGB, LGBM, HistGB) use one core to avoid oversubscription. On PyInstaller Windows builds, parallelization is always disabled.

Partial Dependence

List of features analyzed with partial dependence plots.

Note: Partial Dependence Plots are not available for Neural Networks due to sklearn limitations.

Domain Applicability

Analysis of model applicability domain:

- Training Samples: Number of training samples
- Original Features: Number of input features
- PCA Components: Number of PCA components used
- Explained Variance: Variance captured by PCA
- Thresholds: Leverage, Mahalanobis, k-NN distance thresholds

Prediction Explanation

Explain individual predictions:

120. Enter instance index (row number)
121. Click "Explain Prediction"
122. View:

- Predicted value
- SHAP contributions (top features)
- Top positive/negative contributors

Exporting Results

After running an explainability analysis, click "Export Results as Text" to save all results to a file:

123. Click the "Export Results as Text" button (below "Run Explainability Analysis")
124. Choose a save location (default filename: explainability_{model_name}_{timestamp}.txt)
125. The exported file includes:

- Header: Model name, timestamp, and report title
- Section 1 — Feature Importance: Complete table with Feature, Model Importance, Permutation, SHAP, and Average Rank
- Section 2 — SHAP Values: Global feature importance summary
- Section 3 — Partial Dependence: List of analyzed features (or NN unavailable message)
- Section 4 — Domain Applicability: Training samples, features, PCA components, explained variance, thresholds
- Section 5 — Prediction Explanation: Single-instance explanation (if generated)
- Section 6 — Analysis Log: Complete diagnostic log from the analysis run

Tip: Export after each analysis to keep a permanent record of model interpretation results.

Predict Tab

The Predict Tab allows you to make predictions on new data using saved models.

Loading a Model

126. Click "Load Model Bundle"
127. Select the folder containing:

- manifest.json (required)
- pipeline.joblib or model.pt (required)
- preprocessor.joblib (if applicable)

The model information will be displayed, including:

- Model type and algorithm
- Feature names expected
- Target column name
- Model metrics (if available)
- Validated with (v2.0): the cross-validation strategy used during training, read from the bundle manifest (e.g. "Group K-Fold (by chemical system)"). This is a read-only traceability line — prediction never re-runs CV.

Note on scope: Cross-validation (stratified, grouped, LOO, LOGO) is a training-time evaluation technique. It does not apply to prediction on new data, so the Predict tab has no CV controls — it only reports how the loaded model was validated.

v2.0: Linked Structure Models

When you load a model that was trained with linked crystal or molecule features, the Predict Tab automatically shows the "Link Structures for Prediction (v2.0)" section.

How It Works

128. Auto-detection: The system reads the model's manifest and detects if it requires linked structures (crystal_ or mol_ features)
129. Structure type display: Shows "Crystal (CIF)" or "Molecule (.xyz)" based on the model
130. Validation message: When linked features are missing, the status shows "Linked features required: Use 'Predict with Linked Structures' section below"

Steps for Linked Prediction

131. Load the model bundle (trained with linked features)
132. Load your CSV data (must include an ID column matching structure filenames)
133. In the "Link Structures" section:

- Click "Browse..." to select the folder with .cif or .xyz files
- Select the ID Column from the dropdown (must match structure filenames)
- Verify the Type is correct (auto-detected from model)

134. Click "Predict with Linked Structures"
135. The system will:

- Match CSV IDs with structure files
- Extract structural features
- Auto-generate composition features if the model was trained with them
- Run prediction
- Report: "Successfully linked structures and predicted N rows"

The 2 Main Cases

Case	Structure Type	What You Need	What Happens
A	Crystal (CIF)	CSV with IDs + folder with .cif files	Link .cif by ID → 45 features: 11 structural + 34 periodic compositional (electronegativity, ionization, oxidation states, etc.) → predict
B	Molecule (.xyz)	CSV with IDs + folder	Link .xyz by ID → 149

		with .xyz files	features: composition + geometry 3D + topology + 34 periodic properties + oxidation states → predict

Crystal vs. Molecule — which do I have?

- Crystal = periodic structure with a unit cell (metals, oxides, minerals, semiconductors). File format: .cif primarily
- Molecule = isolated structure without a unit cell (organic molecules, ligands, solvents, gases). File format: .xyz
- Both linkers extract 34 periodic properties (electronegativity, ionization energy, atomic radius, oxidation states, etc.) automatically from the structure file

Loading Structure Files Directly (No CSV)

You can also train directly on structure files without a CSV:

136. Data Tab → Click "Load Molecular Structures" or "Load Molecular Directory" (for .xyz)
137. Data Tab → Click "Load Crystal Structures" or "Load Crystal Directory" (for .cif)
138. Features are extracted automatically — select target and train

What Happens Without Linking

If you click "Run Prediction" (the regular button) with a linked model but without providing structure files:

- An informative dialog appears explaining what's needed
- Shows: "This model was trained with linked [crystal/molecule] features that are missing from the input data"
- Provides the exact steps to use the "Predict with Linked Structures" section
- No crash — graceful error with clear next steps

Loading Prediction Data (Plain Models)

139. Click "Load Prediction Data"
140. Select your CSV or Excel file
141. Data preview will appear

Feature Validation

The system checks if all required features are present:

- Green: All features present
- Blue: Linked features required (use "Predict with Linked Structures")
- Yellow: Missing features can be auto-generated
- Red: Missing features cannot be generated

Auto-Generate Features

If your data has a formula column (e.g., "composition", "formula"):

142. Check "Auto-generate composition features from formula column if missing"
143. The system will generate composition descriptors automatically

Making Predictions (Plain Models)

144. Ensure all required features are present (green status)
145. Click "Run Prediction"
146. Predictions appear in the table

Confidence Intervals

Check "Include Confidence Intervals" to get prediction intervals:

- Regression: Based on CV residual analysis (68% and 95% quantiles)
- Stored as lower_bound and upper_bound columns

Exporting Predictions

147. Click "Export Predictions"
148. Choose save location
149. Predictions are saved as CSV with:

- All original columns
- New prediction column
- lower_bound and upper_bound (if confidence intervals enabled)

Zero Data Leakage Pipeline (v2.0)

This section explains the internal architecture change that makes AutoAIM Studio's cross-validation scientifically valid.

What Was the Problem?

In most AutoML tools (including AutoAIM before v2.0), the typical workflow looks like this:

```
1. Load data
2. Fit StandardScaler on ALL training data
3. Transform training data
4. Run cross-validation on the already-scaled data
```

The bug: Step 2 sees the validation data. When the scaler computes the mean and standard deviation, it includes the validation fold. During CV, that fold is supposed to be "unseen," but its statistics have already leaked into the scaler.

This produces optimistically biased CV scores — the model appears better than it really is because the validation data indirectly influenced the preprocessing.

How AutoAIM v2.0 Fixes It

AutoAIM now uses sklearn Pipeline internally:

```
1. Load data → keep it RAW (no scaling)
2. Build Pipeline([preprocessor, model])
3. Run cross_validate(pipeline, X_raw, y, cv=...)
```

Inside cross_validate(), sklearn does this for every fold:

```
For fold i:
  preprocessor.fit(X_train_fold)   ← scaler sees ONLY training data
  X_train_scaled = preprocessor.transform(X_train_fold)
  model.fit(X_train_scaled, y_train_fold)
  X_val_scaled = preprocessor.transform(X_val_fold)
  score = evaluate(model, X_val_scaled, y_val_fold)
```

Result: The validation fold is truly unseen during both preprocessing and model training.

Where Does This Apply?

Component	Pipeline Used?	Details
Training Tab	Yes	All sklearn models trained with preprocessor parameter
Optimization (Bayesian)	Yes	Each trial wraps estimator in Pipeline before CV
Optimization (Grid Search)	Yes	GridSearchCV runs on Pipeline; param_grid prefixed with model__
Ensemble	Yes	Base models are already Pipelines from Training
Neural Network	Manual	CV loop clones and refits preprocessor per fold before training each PyTorch network
Saved Models	Yes	Exported bundle contains the

		fitted Pipeline (preprocessor + model)
Predict Tab	Yes	Inference receives raw DataFrame; Pipeline applies preprocessing internally

What This Means for Users

You don't need to do anything differently. The GUI looks and works exactly the same. The change is entirely internal:

- Load your data normally
- Select target, preprocess, generate features — same as before
- Train models — CV scores are now valid (no hidden leakage)
- Save models — the bundle contains a complete Pipeline
- Predict — pass raw data; the Pipeline handles scaling automatically

For Developers

If you're extending AutoAIM programmatically:

```
from app.core.data_manager import DataManager
from app.core.trainer import ModelTrainer
from sklearn.ensemble import RandomForestRegressor

dm = DataManager()
dm.load_data("data.csv")
dm.set_target_column("target")

# Returns RAW DataFrames (v2.0)
X_train, X_test, y_train, y_test = dm.prepare_data()

# Get the unfitted preprocessor
preprocessor = dm.get_preprocessor()

# Train with Pipeline (zero data leakage)
trainer = ModelTrainer()
result = trainer.train(
    RandomForestRegressor(), "RF",
    X_train, y_train, X_test, y_test,
    problem_type="regression",
    preprocessor=preprocessor,  # ← key: enables Pipeline
```

```
)

# result.model is a fitted Pipeline
# Accepts raw DataFrames for prediction
preds = result.model.predict(X_test)
```

If you omit preprocessor, the legacy path (numpy arrays, no Pipeline) still works for backward compatibility.

Scientific Impact

This fix ensures that:

150. CV scores reflect true generalization — not an optimistic upper bound
151. Model comparisons are fair — all models evaluated under identical no-leakage conditions
152. Published results are defensible — the methodology follows best practices in ML for materials science
153. Saved models are deployment-ready — the Pipeline encapsulates the complete preprocessing logic

Structure Linking

Structure linking bridges tabular datasets with molecular or crystal structure files, enriching your data with structural descriptors before training. AutoAIM Studio supports two types of structure linking:

Feature	Crystal Structures	Molecular Structures
File format	.cif	.xyz
ID matching	Filename = ID value	Filename = ID value
Features extracted	149 crystal descriptors	149 molecular descriptors
Dependencies	pymatgen	numpy only
Module	CrystalFeatureExtractor	MoleculeFeatureLinker

Crystal Structure Linking (CIF)

How It Works

154. Your dataset has an ID column (e.g., material_id) matching CIF filenames
155. CIF files are named {material_id}.cif in a folder

156. The linker extracts 149 crystal descriptors: lattice (~10), composition (~13), structural (~3), Magpie (106), advanced (~15), derived (~2). Same features as direct CIF loading, with crystal_ prefix.
157. Features are merged into your dataset as crystal_* columns

Pre-Link Check

Before linking, the system reports:

- Matched: How many IDs have corresponding CIF files
- Missing: How many IDs lack CIF files (features filled with NaN)
- Duplicates: Warns if multiple rows share the same ID

Important: Non-Crystalline Detection

The system detects isolated molecules in artificial simulation boxes and warns you that crystal descriptors will be physically meaningless for these structures.

Molecular Structure Linking (.xyz)

How It Works

158. Your dataset has an ID column matching .xyz filenames
159. .xyz files are named {id}.xyz in a folder
160. The linker extracts three levels of molecular descriptors:

Level 1 — Composition (~20 features)

- Atom counts per element (H, C, N, O, S, P, F, Cl, Br, I, Si, B)
- Molecular mass, heavy atom count, elemental diversity
- Element ratios: H/C, O/N, C/N, S/C, P/C, halogen ratios

Level 2 — Geometry 3D (~20 features)

- Center of mass, radius of gyration
- Bounding box: volume, surface, diagonal
- Principal moments of inertia + oblate/prolate shape index
- Asphericity, spherical defect
- Pairwise distance statistics (mean, std, min, max, median)

Level 3 — Topology (~45 features)

- Bond estimation, coordination numbers
- Bond angle statistics + 10-bin histogram
- Ring detection (3-8 membered) + ring size histogram
- Functional group detection: OH, NH, C=O, C=C, C≡C, C≡N, C-F, C-Cl, C-Br, PO, CS, SO
- Bond type ratios, saturation/unsaturation indices

Advantages

- Zero external dependencies: Pure numpy, no RDKit
- Numeric prefix tolerant: Handles .xyz files with prefixed elements (e.g., "1C 0.0 0.0 0.0")
- Missing structures filled with NaN (trainable after imputation)

Predicting with Linked-Structure Models

The Problem

If you train a model with linked crystal/molecule features and later try to predict on data without those features, the system detects this and provides an actionable error instead of crashing.

The Solution

When a model requiring linked structures is loaded:

161. The Predict Tab shows a "Link Structures" section
162. Select your structure folder and ID column
163. Click "Predict with Linked Structures"

The system automatically:

164. Matches IDs with structure files
165. Extracts structural features
166. Merges them into your data
167. Runs prediction

Programmatic Usage

```
from app.core.inference_engine import InferenceEngine

engine = InferenceEngine("path/to/model_bundle")

# Check if model requires linked structures
if engine.is_structure_linked_model():
    reqs = engine.get_linked_structure_requirements()
    print(f"Requires: {reqs['structure_type']} features")
    print(f"Missing: {reqs['missing_feature_count']} features")

# Predict with automatic linking
result = engine.predict_with_linked_structures(
    df=new_data,
    id_column="material_id",
    structure_folder="/path/to/cif_or_xyz_files/"
```

```
    # structure_type auto-detected from manifest
)
```

What Happens Without Linking

If you call predict() without linked features:

- Informative error: "This model requires linked crystal features... Use predict_with_linked_structures()"
- Example code: Shows exact syntax needed
- No crash: Graceful error with clear next steps

Troubleshooting

Common Issues

"Cannot generate missing features automatically"

Cause: Missing composition features and no formula column detected

Solution:

168. Ensure your prediction data has a column with chemical formulas
169. Check the "Auto-generate composition features" checkbox
170. The column name should contain "formula", "composition", or "structure"

"Selected model does not have predict method"

Cause: Model object is corrupted or not properly loaded

Solution:

171. Retrain the model
172. Check if model was saved correctly
173. For Neural Networks, ensure model was built before training

"Model 'X' not found in any source"

Cause: Model was deleted or not properly stored

Solution:

174. Retrain the model
175. Save the model again after training

"SHAP values not available for this model"

Cause: SHAP computation failed (usually for Neural Networks)

Solution:

176. Ensure model has a working predict method
177. Try with a smaller dataset sample
178. Check console for detailed error messages

"Partial dependence plots not available"

Cause: PDP is not supported for Neural Networks

Solution:

- This is expected behavior. Use Feature Importance and SHAP tabs instead.

Cross-Validation Strategy Issues (v2.0)

"Grouping produced N group(s) but K folds were requested"

Cause: With Group K-Fold, you asked for more folds than there are groups.

Solution: Reduce the CV Folds to a number ≤ the number of groups, or choose a different grouping (e.g. "Exact formula" or a higher cluster count produces more groups than "Chemical system").

"Leave-One-Group-Out needs at least 3 groups"

Cause: LOGO requires at least 3 distinct groups.

Solution: Use Group K-Fold instead, or pick a grouping that yields more groups.

"N sample(s) have no value in column 'X'"

Cause: You chose "Dataset column" grouping, but that column has missing values.

Solution: Fill or remove those rows first. AutoAIM never silently drops samples.

"No composition/formula column was detected"

Cause: You chose a chemical grouping source (chemical system, formula, or clustering), but no column in your dataset looks like chemical formulas.

Solution: Add a formula/composition column, or use "Dataset column" grouping.

"Could not build quantile bins..."

Cause: Stratified K-Fold couldn't bin the target so every fold sees every bin (target nearly constant, or too few samples for the requested folds).

Solution: Reduce CV Folds, or use standard K-Fold. If the target is constant, stratification is impossible.

"Cannot reserve a bin for X: only N sample(s) match..."

Cause: You reserved a bin for a special value, but fewer samples match it than the number of folds, so the value cannot appear in every fold.

Solution: Increase the Tolerance (if your data stores the value as e.g. 0.01 instead of 0), reduce CV Folds, or turn off the reserved bin and use plain quantile stratification.

Reserved bin warning: "matches only N of M samples (X%)"

Cause: Your special value is a small fraction of the data (<5%), so the reserved bin may have little effect — or it may not be a true 'property absent' marker.

Solution: Confirm the value is a genuine degenerate marker (e.g. Tc = 0 K for non-superconductors, Eg = 0 eV for metals). If it is, proceed; if not, disable the reserved bin.

Performance Tips

Speed Up Training

179. Reduce CV folds: Use 3 instead of 5
180. Use GPU: Enable GPU acceleration for Neural Networks
181. Reduce trials: Use fewer optimization trials (20-30 instead of 50+)
182. Sample data: Train on a subset for initial exploration

Improve Accuracy

183. Generate composition features: Use all 106 descriptors
184. Optimize hyperparameters: Run optimization with more trials
185. Create ensembles: Combine multiple models
186. Try different algorithms: Not all algorithms work equally well for all datasets

Reduce Memory Usage

187. Disable composition features: If not needed
188. Reduce batch size: For Neural Networks
189. Use fewer optimization trials: Each trial keeps results in memory

Getting Help

If you encounter issues not covered here:

190. Check the log output in each tab

191. Review the console/terminal for detailed error messages
192. Ensure your data is properly formatted (CSV/Excel)
193. Verify all required columns are present

Feature Reference

Composition Features (106 Descriptors)

The custom composition feature generator computes the following statistics for each chemical formula:

Atomic Properties:

- Atomic number (mean, std, min, max, range, median, percentiles)
- Atomic mass (mean, std, min, max, range, median, percentiles)
- Covalent radius (mean, std, min, max, range, median, percentiles)
- Van der Waals radius (mean, std, min, max, range, median, percentiles)

Electronic Properties:

- Electronegativity (mean, std, min, max, range, median, percentiles)
- Ionization energy (mean, std, min, max, range, median, percentiles)
- Electron affinity (mean, std, min, max, range, median, percentiles)

Physical Properties:

- Density (mean, std, min, max, range, median, percentiles)
- Boiling point (mean, std, min, max, range, median, percentiles)
- Melting point (mean, std, min, max, range, median, percentiles)

Periodic Table Properties:

- Group number (mean, std, min, max, range, median, percentiles)
- Period number (mean, std, min, max, range, median, percentiles)

Additional Features:

- Number of unique elements
- Total number of atoms
- Element fractions

These features are computed from elemental properties and provide a comprehensive representation of material composition for machine learning models.

Appendix: Model Algorithms

Random Forest

- Type: Ensemble of decision trees

- Strengths: Handles non-linear relationships, robust to outliers
- Best for: General-purpose regression, feature importance

Gradient Boosting

- Type: Sequential ensemble
- Strengths: High accuracy, handles complex patterns
- Best for: Competitive accuracy, medium-sized datasets

XGBoost

- Type: Optimized gradient boosting
- Strengths: Fast, regularized, handles missing values
- Best for: Tabular data competitions, production systems

LightGBM

- Type: Histogram-based gradient boosting
- Strengths: Very fast, memory efficient
- Best for: Large datasets, speed-critical applications

CatBoost

- Type: Gradient boosting with ordered boosting
- Strengths: Excellent categorical handling, reduced overfitting
- Best for: Datasets with many categorical features

SVR (Support Vector Regression)

- Type: Kernel-based method
- Strengths: Effective in high dimensions, memory efficient
- Best for: Small to medium datasets, non-linear relationships

Ridge Regression

- Type: Linear regression with L2 regularization
- Strengths: Simple, fast, prevents overfitting
- Best for: Baseline model, linear relationships

Neural Networks

- Type: Deep learning with customizable architecture
- Strengths: Learns complex non-linear patterns
- Best for: Large datasets, complex feature interactions

Feature Documentation

For the complete list of every descriptor computed by AutoAIM, see the dedicated feature reference documents:

Document	Features	Description
FEATURES_CRYSTAL.md	149	Crystal structure descriptors (lattice, composition, Magpie, structural, advanced)
FEATURES_MOLECULAR.md	149	Molecular descriptors (composition, geometry 3D, topology, oxidation states)
FEATURES_COMPOSITION.md	106	Magpie composition statistics (13 properties × 8 stats + 2 derived)

AutoAIM Studio - Materials Science Machine Learning Platform

Version 2.0.0 - User Manual

Citation

If you use AutoAIM Studio in your research, please cite:

```
@article{vera2026autoaim,
  title = {AutoAIM Studio: A desktop application for automated
machine learning in materials science},
  author = {Vera De la Garza, G. V. and Fomine, S.},
  journal = {SoftwareX},
  volume = {35},
  pages = {102945},
  year = {2026},
  doi = {10.1016/j.softx.2026.102945},
  url = {https://github.com/cesargabrielvera1-ux/AutoAIM-Studio}
}
```

Contact: veradelagarza@quimica.unam.mx

DOI: 10.1016/j.softx.2026.102945